

Panoramica comparativa delle soluzioni NoSQL

Famiglie tecnologiche, scenari applicativi e trade-off

NoSQL

Materiale aggiuntivo per blocchi 13-16

1. Perché una panoramica comparativa

NoSQL non è una singola tecnologia: è una famiglia di risposte diverse a problemi diversi.

Problema

Dopo SQL è facile dire “usiamo NoSQL” come se fosse una scelta unica. In realtà bisogna scegliere tra modelli dati, garanzie, query, costi operativi e scenari d'uso molto diversi.

Idea guida

La domanda corretta non è “qual è il database migliore?”, ma “quale modello serve meglio questo access pattern, con quali garanzie e quali costi?”.

NoSQL come insieme di specializzazioni

- ▶ Document database: documenti JSON/BSON con schema flessibile.
- ▶ Key-value store: accesso rapido tramite chiave.
- ▶ Wide-column store: scritture distribuite ad alto volume.
- ▶ Graph database: relazioni profonde come elemento centrale.
- ▶ Search engine: ricerca testuale, log analytics, ranking.
- ▶ Time-series database: dati misurati nel tempo, ingest continuo e finestre temporali.
- ▶ Vector database: similarità semantica tra embedding.

Domanda	Effetto sulla scelta
Come leggo il dato? Quanto scrivo?	per chiave, documento, tempo, testo, relazione, vettore ingest continuo, burst, scritture distribuite, aggiornamenti frequenti
Quanto devo essere coerente?	transazione forte, coerenza eventuale, read-your-writes
Come cresco?	singola macchina, replica, sharding, multi-region
Chi lo opera?	self-managed, managed cloud, competenze disponibili
Quanto costa sbagliare?	perdita dati, dashboard lenta, inconsistenza temporanea

Matrice sintetica delle famiglie

Famiglia	Unità naturale	Query tipica	Esempi di scenario
Document	documento JSON	leggi/modifica aggregate	profili, cataloghi, eventi semi-strutturati
Key-value	coppia chiave-valore	lookup diretto	sessioni, cache, carrelli, feature flag
Wide-column	partizione + colonne	range dentro partizione	timeline, metriche massive, log operativi
Graph	nodo + relazione	attraversamento	frodi, dipendenze, reti, raccomandazioni
Search	documento indicizzato	full-text, ranking, facet	knowledge base, log search, audit
Time-series	misura timestamped	finestre temporali	monitoring, IoT, energia, SRE
Vector	embedding	nearest neighbors	ricerca semantica, RAG, deduplicazione

2. Document database

Il documento è utile quando il dato viene letto e scritto come aggregate coerente.

Un database documentale memorizza record come documenti strutturati, spesso simili a JSON. Il documento può contenere campi annidati e array.

Esempio

Un ordine può contenere testata, indirizzo snapshot, righe ordine e stato di pagamento nello stesso documento, se vengono letti insieme.

Esempio documento

```
{
  "order_id": "ORD-2026-1001",
  "customer": { "id": "C-42", "segment": "business" },
  "status": "shipped",
  "items": [
    { "sku": "BAT-10", "qty": 3, "unit_price": 129.0 },
    { "sku": "INV-02", "qty": 1, "unit_price": 880.0 }
  ],
  "shipping": { "city": "Roma", "carrier": "field-logistics" }
}
```

Quando funziona bene

- ▶ L'applicazione legge quasi sempre l'oggetto completo.
- ▶ La struttura ha campi opzionali o versioni diverse.
- ▶ Le relazioni non richiedono join complessi e frequenti.
- ▶ Lo schema evolve spesso, ma gli access pattern sono noti.
- ▶ Si vuole conservare la forma naturale di messaggi JSON o eventi.

Rischio	Sintomo
documenti troppo grandi	aggiornamenti costosi, duplicazione eccessiva
relazioni nascoste	join ricostruiti a mano nel codice applicativo
schema non governato	campi simili con nomi diversi
query ad hoc complesse	pipeline difficili da leggere e ottimizzare
duplicazione non controllata	valori divergenti tra documenti

3. Key-value e cache

Quando l'accesso è per chiave, il modello più semplice può essere anche il più efficace.

Idea

Ogni valore è recuperato tramite una chiave nota. Il database non deve capire molto del contenuto: deve rispondere velocemente.

- ▶ sessioni utente;
- ▶ token temporanei;
- ▶ carrelli e stato volatile;
- ▶ cache di risposte costose;
- ▶ contatori e rate limiting;
- ▶ feature flag e configurazioni runtime.

Key-value: esempio operativo

Chiave	Valore
<code>session:user:928</code>	dati sessione con scadenza
<code>cart:user:928</code>	carrello corrente
<code>rate:api:client-77</code>	contatore richieste/minuto
<code>feature:new-dashboard</code>	flag attivo/disattivo
<code>kpi:region:north:today</code>	KPI precomputato

- ▶ Molto efficace per lookup diretti e dati temporanei.
- ▶ Poco adatto a interrogazioni esplorative.
- ▶ Spesso usato insieme a un database primario, non al suo posto.
- ▶ Introduce problemi di invalidazione cache: quando il dato cambia, chi aggiorna la copia?
- ▶ Nei sistemi distribuiti può diventare parte critica della disponibilità.

4. Wide-column

Per volumi molto grandi e scritture distribuite, il modello parte dalla partizione e dalla query.

Nei sistemi wide-column il dato è organizzato per partizioni. La progettazione parte dalla domanda: quale chiave raggruppa le righe che leggerò insieme?

Esempio

Per una timeline eventi, una partizione può essere `device_id + giorno`; dentro la partizione ordino per timestamp.

Esempio modello query-first

```
CREATE TABLE readings_by_device_day (  
  device_id text,  
  day date,  
  ts timestamp,  
  value double,  
  status text,  
  PRIMARY KEY ((device_id, day), ts)  
);
```

La chiave primaria non descrive solo identità: descrive anche come il dato sarà distribuito e letto.

Quando scegliere wide-column

- ▶ scritture continue ad alto volume;
- ▶ query note e ripetitive;
- ▶ scala orizzontale e alta disponibilità;
- ▶ dati append-only o quasi append-only;
- ▶ accettazione di compromessi sulla flessibilità delle query.

Rischio	Esempio
partition hot spot	troppi eventi sulla stessa chiave
query non prevista	serve una nuova tabella denormalizzata
consistenza eventuale	letture temporaneamente non allineate
modello duplicato	stessa informazione in più viste fisiche
operatività distribuita	tuning, repair, compaction, capacity planning

5. Graph database

Se la domanda riguarda cammini, dipendenze e connessioni, la relazione diventa il dato primario.

Graph database: idea

Un graph database rappresenta la realtà come nodi, relazioni e proprietà.

Domande naturali

Quali clienti sono collegati allo stesso contatore? Quali asset dipendono da una cabina? Quali fornitori condividono indirizzi, dispositivi o contatti?

- ▶ frodi e collusioni;
- ▶ reti elettriche e dipendenze asset;
- ▶ raccomandazioni;
- ▶ knowledge graph documentale;
- ▶ controllo accessi basato su relazioni;
- ▶ impatto di un guasto lungo una rete.

Graph: quando non serve

- ▶ quando le relazioni sono semplici e poco profonde;
- ▶ quando servono soprattutto aggregazioni tabellari;
- ▶ quando il team non ha competenze sul modello a grafo;
- ▶ quando si vuole usare un graph database solo per evitare join ordinari;
- ▶ quando il dato è massivamente temporale e le relazioni sono secondarie.

6. Search e log analytics

Quando la domanda è testuale o esplorativa, l'indice invertito diventa centrale.

Idea

Un motore di ricerca indicizza documenti per trovare testo, filtri, facet, ranking e pattern quasi in tempo reale.

- ▶ ricerca in manuali, ticket, contratti e report;
- ▶ log analytics;
- ▶ audit e investigazioni operative;
- ▶ observability applicativa;
- ▶ ricerca geospaziale o per attributi testuali.

Richiesta	Perché search è utile
“trova ticket simili”	ranking testuale, sinonimi, scoring
“filtra log per errore e servizio”	ingest e query su messaggi semi-strutturati
“cerca manuali su inverter”	full-text e facet per categoria/prodotto
“mostra eventi anomali”	aggregazioni veloci su indice

7. Time-series e vector

Alcuni problemi sono così ricorrenti da meritare motori specializzati.

Time-series database

Un time-series database ottimizza ingest, compressione, retention e query su finestre temporali.

- ▶ metriche infrastrutturali;
- ▶ misure industriali;
- ▶ energia e consumi;
- ▶ monitoring e alerting;
- ▶ downsampling e retention differenziata.

Il laboratorio telemetria usa MongoDB per didattica documentale; in produzione, per certi carichi, si valuterebbe anche un time-series database.

Idea

Un vector database cerca oggetti simili nello spazio degli embedding, non righe uguali o documenti con parole identiche.

- ▶ ricerca semantica in documenti tecnici;
- ▶ RAG su knowledge base;
- ▶ deduplicazione di descrizioni simili;
- ▶ raccomandazioni;
- ▶ clustering di ticket e incidenti.

8. Scenari applicativi

La scelta del database diventa più chiara se partiamo da casi concreti.

Tabella scenario-tecnologia

Scenario	Tecnologia candidata	Motivo
catalogo prodotti variabile	document database	campi diversi per categoria
sessioni web	key-value/cache	lookup rapido e scadenza
timeline eventi massive	wide-column	scritture distribuite e query note
dipendenze asset	graph database	attraversamento relazioni
manuali e ticket testuali	search engine	full-text e ranking
metriche ogni secondo	time-series	retention e finestre temporali
ricerca semantica documenti	vector database	similarità di embedding

Caso 1: catalogo energetico

Problema

Gestire prodotti e servizi energetici con attributi diversi: pannelli, inverter, batterie, contratti, servizi di manutenzione.

- ▶ Document DB per schede prodotto flessibili.
- ▶ Search engine per ricerca testuale e filtri.
- ▶ Relazionale per prezzi, contratti, ordini e fatturazione.
- ▶ Cache per disponibilità e configurazioni lette spesso.

Problema

Trovare rapidamente ticket simili, documenti tecnici, storico cliente e possibili escalation.

- ▶ Relazionale per ticket, SLA e stati ufficiali.
- ▶ Search per testi, note, email e manuali.
- ▶ Vector search per similarità semantica tra problemi.
- ▶ Graph per legami cliente-impianto-asset-fornitore.

Caso 3: fraud e anomalie contrattuali

Problema

Identificare pattern non evidenti in contratti, pagamenti, indirizzi, dispositivi e referenti.

- ▶ Graph per connessioni indirette e comunità sospette.
- ▶ Relazionale per record contrattuali e pagamenti ufficiali.
- ▶ Search per note e documenti allegati.
- ▶ Feature store o analytical store per modelli di scoring.

Caso 4: observability piattaforma

Problema

Monitorare microservizi, log, metriche, errori e alert di un sistema applicativo.

- ▶ Time-series per metriche numeriche nel tempo.
- ▶ Search/log analytics per log applicativi.
- ▶ Key-value per rate limit e stato temporaneo.
- ▶ Dashboard per SRE e team applicativi.

9. Criteri decisionali

Una scelta difendibile combina requisiti funzionali, requisiti non funzionali e capacità operative.

Checklist di scelta

Criterio	Domanda da porre
Access pattern	quali query devono essere veloci sempre?
Forma del dato	tabellare, documento, grafo, testo, tempo, vettore?
Scrittura	append-only, update frequente, burst, stream?
Consistenza	posso accettare ritardi o divergenze temporanee?
Operatività	chi gestisce backup, replica, upgrade, monitoraggio?
Costo	pago storage, richieste, nodi, licenze, competenze?
Evoluzione	quanto cambieranno schema, query e integrazioni?

Anti-pattern frequenti

- ▶ scegliere NoSQL perché “scala”, senza misurare il problema reale;
- ▶ usare documenti come tabelle senza ripensare gli access pattern;
- ▶ duplicare dati senza decidere quale copia è autorevole;
- ▶ ignorare consistenza e ritardi di propagazione;
- ▶ sottovalutare backup, osservabilità e migrazioni;
- ▶ introdurre troppe tecnologie prima di avere un bisogno chiaro.

- ▶ SQL resta essenziale per molti dati core.
- ▶ NoSQL non è un sostituto generale, ma un insieme di strumenti specializzati.
- ▶ Ogni tecnologia porta benefici e costi.
- ▶ A scala alta la domanda non è “quale DB?”, ma “quale architettura dati serve ai diversi access pattern?”.

- ▶ MongoDB Manual, Data Modeling: mongodb.com/docs/manual/data-modeling/
- ▶ Apache Cassandra Documentation, Architecture Overview: cassandra.apache.org/doc/stable/
- ▶ Redis Docs, Data types: redis.io/docs/latest/develop/data-types/
- ▶ Neo4j Docs, Graph database concepts: neo4j.com/docs/getting-started/graph-database/
- ▶ AWS DynamoDB Developer Guide: docs.aws.amazon.com/amazondynamodb/
- ▶ Elastic Docs, Elasticsearch Reference: elastic.co/docs/reference/elasticsearch
- ▶ InfluxData Platform Docs: docs.influxdata.com/platform/